

«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)

ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ

Добавлено примечание ([МОЭВМ1]):

Добавлено примечание ([2]):

Тема: UI-тестирование

Студенты гр.
4381

А.В. Захаренко
К.А. Аникин
Д.Е. Бабин
А.В. Землякова

Руководитель

А.М. Шевелёва

Санкт-Петербург
2026

**ЗАДАНИЕ
НА УЧЕБНУЮ ПРАКТИКУ**

Студент: А.В. Землякова

Группа: 4381

Тема практики: UI - тестирование

Задание на практику:

Нужно написать 10 тестов на команду по одному определенному блоку / функционалу системы.

Выбирайте систему для тестирования функционала.

Каждый тест должен состоять минимум из 5 действий. Итоговое минимально кол-во всех действий – 50. Если за 10 тестов вы их не набираете, придется писать дополнительные тесты. При этом авторизация не считается действием, а любая проверка считается 1 действием.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студент	_____	А.В. Землякова
---------	-------	----------------

Руководитель	_____	А.М. Шевелева
--------------	-------	---------------

АННОТАЦИЯ

В данном отчете представлены результаты прохождения учебной практики по направлению «UI-тестирование». В рамках практики был разработан набор автоматизированных тестов для веб-сайта Aviasales. Тесты написаны на языке Java с использованием фреймворков Selenide и JUnit, архитектура построена на основе паттерна Page Object Model. Разработано 10 тестов, покрывающих функциональность поиска билетов, изменения количества пассажиров, выбора класса обслуживания, смены валюты, применения фильтров и поиска отелей. Отчет содержит описание поставленных задач, используемых технологий, архитектуры проекта и результатов тестирования.

SUMMARY

This report presents the results of a training internship in the field of "UI testing". As part of the practice, a set of automated tests was developed for the Aviasales website. The tests are written in Java using Selenide and JUnit frameworks, the architecture is based on the Page Object Model pattern. 10 tests have been developed covering the functionality of ticket search, changing the number of passengers, choosing a class of service, changing the currency, applying filters and searching for hotels. The report contains a description of the tasks set, the technologies used, the project architecture, and the test results.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ (заголовок второго уровня)5

6

6

6

1.3 Третий раздел первой главы (заголовок третьего уровня)6

Ошибка! Закладка не определена.

12

Ошибка! Закладка не определена.

Ошибка! Закладка не определена.

16

18

19

20

21

ВВЕДЕНИЕ

В рамках учебной практики по направлению «UI-тестирование» была поставлена задача разработать набор автоматизированных тестов для веб-приложения Aviasales (<https://www.aviasales.ru>). Aviasales — это популярный сервис для поиска и бронирования авиабилетов, отелей и других туристических услуг. Актуальность работы обусловлена необходимостью автоматизации тестирования для повышения качества продукта и сокращения времени на регрессионное тестирование.

Целью практики является освоение технологий автоматизированного UI-тестирования и разработка рабочего набора тестов для реального веб-приложения.

Задачи практики:

1. Изучить технологии Java, Selenide, JUnit, Maven.
2. Освоить паттерн Page Object Model для организации кода тестов.
3. Разработать чек-лист тестов для Aviasales.
4. Реализовать 10 автоматизированных тестов.
5. Оформить отчет по результатам практики.

Вклад участников:

Аникин К.А. (тимлид) — разработка базовой архитектуры проекта, создание классов BaseElement, BasePage, BaseTest, реализация тестов 1-3 (поиск билетов).

Захаренко А.В. — реализация тестов 4-6 (выбор бизнес-класса, изменение количества пассажиров, смена валюты), доработка элементов страниц.

Бабин Д.Е. — реализация тестов 7-9 (поиск отелей и фильтрация).

Землякова А.В. — реализация теста 10 (поиск отелей с фильтрацией).

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

UI-тестирование (User Interface Testing) — это процесс проверки пользовательского интерфейса программного продукта на соответствие требованиям и ожиданиям пользователей. В современных веб-приложениях автоматизация UI-тестов позволяет значительно сократить время на регрессионное тестирование, уменьшить количество человеческих ошибок и обеспечить стабильность продукта при частых изменениях кода [1].

В рамках практики объектом тестирования выбран сайт Aviasales — один из крупнейших сервисов по поиску авиабилетов и отелей в России. Автоматизация тестирования для таких сервисов особенно актуальна, так как изменения на сайте происходят часто, а любой сбой в работе может привести к финансовым потерям.

1.2 Задачи практики

В ходе прохождения учебной практики перед командой были поставлены следующие задачи:

1. Изучить теоретические основы UI-тестирования и паттерна Page Object Model.
2. Освоить среду разработки IntelliJ IDEA, систему сборки Maven, язык Java.
3. Изучить библиотеки Selenide для UI-тестов и JUnit для написания тестов.
4. Изучить систему контроля версий Git и работу с репозиториями на GitHub.
5. Разработать чек-лист из 10 тестов для сайта Aviasales.
6. Реализовать автоматизированные тесты.
7. Оформить отчет по результатам практики.

1.3 Третий раздел первой главы

В рамках практики командой был разработан набор из 10 автоматизированных UI-тестов для сайта Aviasales. Каждый тест проверяет

отдельный сценарий использования системы. Ниже приведено краткое описание каждого теста.

Тест №1. Поиск авиабилетов в одну сторону.

Данный тест проверяет базовый сценарий поиска авиабилетов. Тест заполняет поля "Откуда", "Куда" и "Когда" (завтрашняя дата), затем нажимает кнопку поиска. Ожидаемый результат: переход на страницу результатов и отображение списка найденных рейсов с указанием выбранных городов и даты.

Тест №2. Поиск авиабилетов туда-обратно.

Тест проверяет сценарий поиска билетов с обратной датой. Заполняются поля "Откуда", "Куда", "Когда" (завтра) и "Обратно" (послезавтра). Ожидаемый результат: переход на страницу результатов с отображением выбранных городов и обеих дат.

Тест №3. Поиск билетов без города прибытия.

Тест проверяет валидацию формы поиска. Заполняется только поле "Откуда" и даты, поле "Куда" оставляется пустым. Ожидаемый результат: появление сообщения об ошибке "Укажите город прибытия".

Тест №4. Выбор бизнес-класса.

Тест проверяет возможность выбора класса обслуживания. Открывается окно выбора класса, выбирается опция "Бизнес". Ожидаемый результат: в параметрах поиска отображается выбранный класс "Бизнес".

Тест №5. Изменение количества пассажиров.

Тест проверяет возможность изменения количества взрослых пассажиров. Открывается окно пассажиров, количество взрослых увеличивается с 1 до 2. Ожидаемый результат: в параметрах поиска отображается "2 пассажира".

Тест №6. Изменение валюты.

Тест проверяет смену валюты с RUB на USD. Через меню выбора локали выбирается USD. Ожидаемый результат: цены на билеты отображаются с символом \$.

Тест №7. Поиск отелей.

Тест проверяет базовый сценарий поиска отелей. Заполняются поля "Город", "Дата заезда" (завтра), "Дата выезда" (послезавтра) и "Гости" (2). Ожидаемый результат: отображение списка отелей в выбранном городе.

Тест №8. Поиск отелей не дороже 10000 рублей.

Тест проверяет фильтрацию отелей по максимальной цене. После поиска отелей в поле "Максимальная цена" вводится значение 10000. Ожидаемый результат: отображение отелей с ценой не выше 10000 рублей.

Тест №9. Сортировка отелей по убыванию цены.

Тест проверяет сортировку результатов поиска отелей. После поиска применяется фильтр сортировки "По убыванию цены". Ожидаемый результат: отели отсортированы от дорогих к дешевым.

Тест №10. Просмотр предложения самого дешевого отеля.

Тест проверяет переход на страницу предложения отеля. После поиска отелей выбирается самый дешевый отель из списка. Ожидаемый результат: открытие страницы с подробной информацией об отеле.

Для решения поставленных задач использовался паттерн Page Object Model [1]. Паттерн POM говорит нам, что каждый новый класс представляет новую страницу и классы тест-кейсов работают с этими страницами. Классы тест-кейсов взаимодействует с классами страниц, а классы страниц – с классами элементов страниц. Для взаимодействия со страницей использовался Selinide [2], для тестирования использовался Junit [3].

Подробное описание структуры кода представлены на uml — диаграмме (рис. 0)

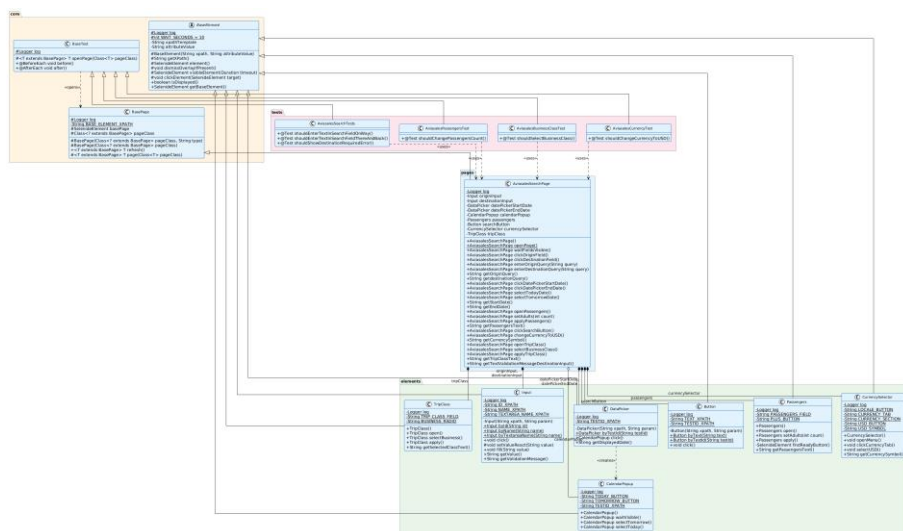


Рисунок 0 - uml — структуры кода

Далее идет описание базовых классов:

- BaseElement — базовый класс элементов страниц

Методы:

- BaseElement — инициализирует класс
- getXPath — возвращает Xpath которым инициализировали класс
- element — возвращает значение элемента по его Xpath
- dismissOverlayIfPresent — закрывает оверлей с умными ценами

- `visibleElement` — ожидает появления элемента
- `clickElement` — производит нажатие по элементу
- `isDisplayed` — проверяет отображается ли элемент
- `getBaseElement` — возвращает значение элемента
- `BasePage` — базовый класс страница

Методы:

- `BasePage` — инициализирует класс
- `refresh` — обновляет стрингу
- `page` — создает и возвращает элемент страницы по её классу
- `BaseTest` — базовый класс тестов

Методы:

- `openPage` — открывает страницу для тестирования
- `before` — задает настройки драйвера сайта для тестирования
- `after` — закрывает драйвер

Далее идет описание классов элементов:

- `Button` — класс кнопки

Методы:

- `BaseElement` — инициализирует класс классами
- `CalendarPopup` — класс всплывающего календаря

Методы:

- `CalendarPopup` — инициализирует класс используя базовый конструктор
- `waitVisible` — запускает ожидание до появления элемента на странице
- `selectTomorrow` — выбирает завтрашнюю дату
- `selectToday` — выбирает сегодняшнюю дату

- `CurrencySelector` — класс селектора

Методы:

- BaseElement — инициализирует класс классами
- DatePicker — класс поля даты

Методы:

- byTestId — инициализирует класс используя id
- click — нажимает по элементу
- getDisplayedDate — возвращает значение из поле даты
- Input — класс поля для ввода текста

Методы:

- Input — инициализирует класс
- byId — инициализирует класс используя id
- byName — инициализирует класс используя name
- byTextareaName — инициализирует класс используя textarea
- click — нажимает на поле ввода
- setValueReact — устанавливает значение в поле используя javascript (используется для метода fill)
- fill — заполняет поле input
- getValue — возвращает значение из поля
- Passengers — класс элемента выбора количества пассажиров

Методы:

- BaseElement — инициализирует класс классами
- TripClass — класс элемента выбора класса билета

Методы:

- BaseElement — инициализирует класс классами

Далее идет описание классов страниц:

- AviasalesSearchPage — класс главной страницы поиска авиасейлс

Методы:

- shouldEnterTextInSearchFieldOnWay — метод тестирования поиска билетов в один конец

- `shouldEnterTextInSearchFieldThereAndBack` — метод тестирования поиска билетов туда обратно
- `shouldShowDestinationRequiredError` — метод тестирования вывода предупреждения о не заполненности поля «куда»

Далее идет описание классов тестов:

- `AviasalesBusinessClassTest`

Методы:

- `BaseElement` — инициализирует класс классами

- `AviasalesCurrencyTest`

Методы:

- `BaseElement` — инициализирует класс классами

- `AviasalesPassengersTest`

Методы:

- `BaseElement` — инициализирует класс классами

- `AviasalesSearchTests` класс тестов тестирования поиска билетов

Методы:

- `BaseElement` — инициализирует класс классами

2.1. Просмотр предложения самого дешевого отеля

В данном тесте проверяется корректность открытия страницы самого дешевого предложения отеля после поиска и сортировки результатов по возрастанию цены путем заполнения полей «Куда», «Заезд», «Выезд», «Количество человек».

План выполнения теста:

- Нажатие на кнопку выбора раздела по поиску отелей
- Нажатие на поле куда отправитесь и выбрать Москва
- Нажатие на поле дата заезда и выбрать завтра
- Нажатие на поле дата выезда и выбрать послезавтра
- Нажатие на ввод количества гостей и выбое цифры 2

- Нажатие на кнопку найти отели и дождаться перехода на страницу с отелями
- Нажатие на фильтр упорядочивания выбор фильтра цены по возрастанию
- Нажатие на самое первое предложение в результатах поиска и проверка корректности открытия страницы выбранного отеля.

Пример работы теста от лица пользователя с фиксацией каждого действия.

Открытие страницы Авиасейлс и выбор раздела отелей (рис. 1).

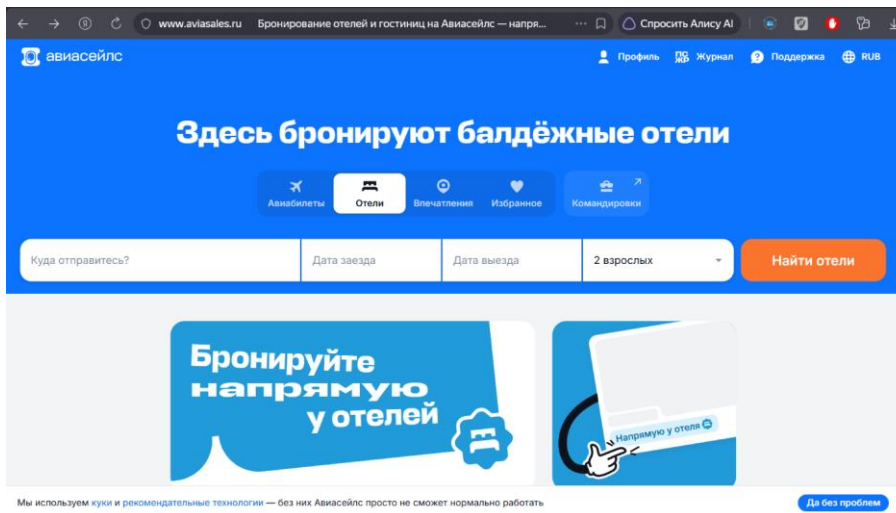


Рисунок 1 - Открытие страницы Авиасейлс

Нажатие на поле «Куда» отправитесь и выбрать Москва (рис. 2).

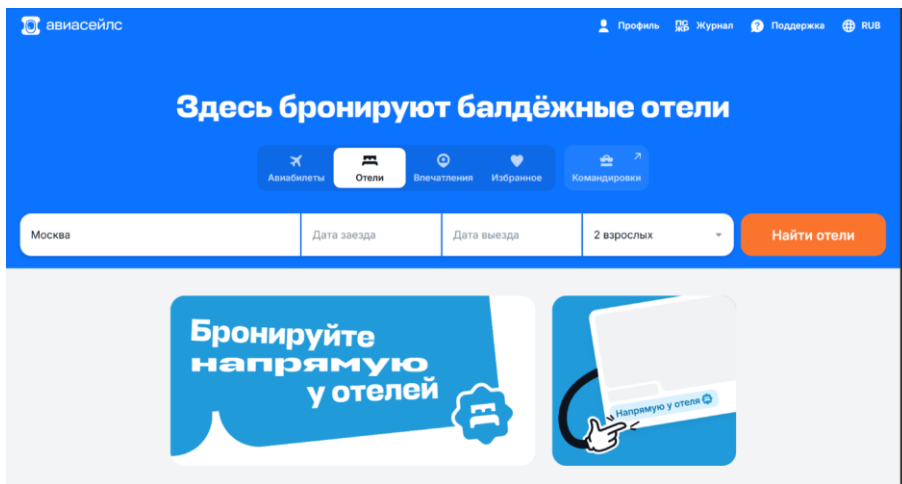


Рисунок 2 - Вставка в поле "Куда"

Нажатие на поле дата заезда и выбрать завтра (рис. 3)

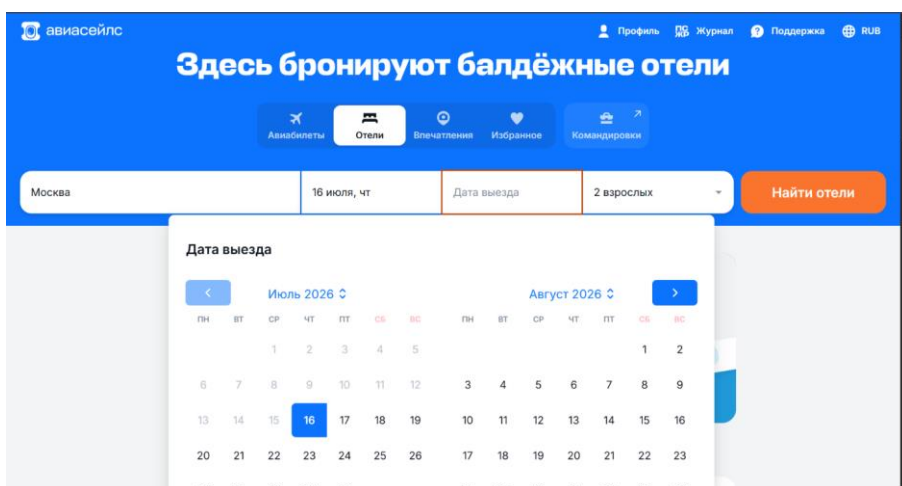


Рисунок 3 - Вставка в поле "Заезд"

Нажатие на поле дата выезда и выбрать послезавтра (рис. 4).

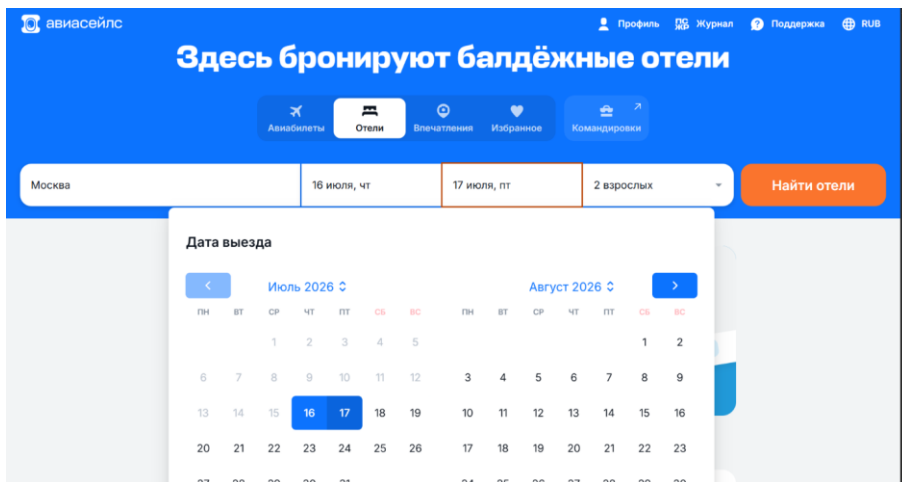


Рисунок 4 - Нажатие на поле "Выезд"

Нажатие на ввод количества гостей и выбое цифры 2 (рис. 5)

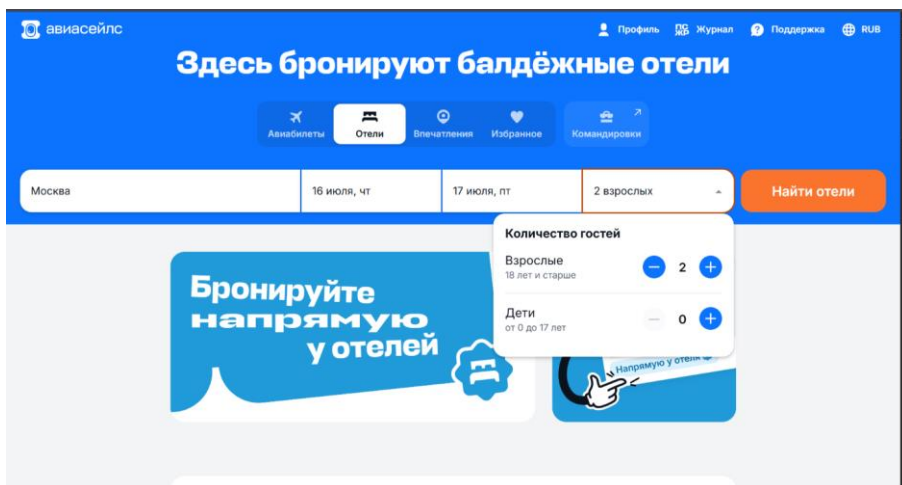


Рисунок 5 – Выбор количества гостей

Нажатие на кнопку найти отели и дождаться перехода на страницу с отелями (рис. 6)

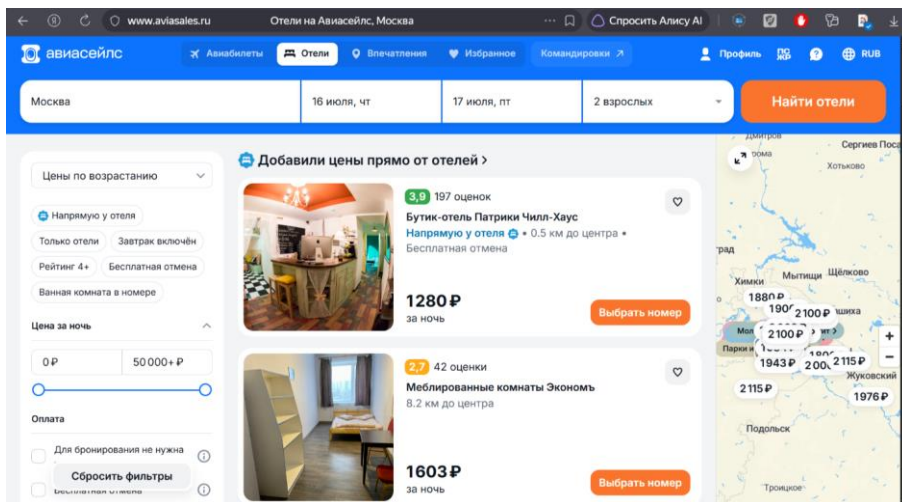


Рисунок 6 – Переход на страницу с отелями

Нажатие на самое первое предложение в результатах поиска (рис. 7).

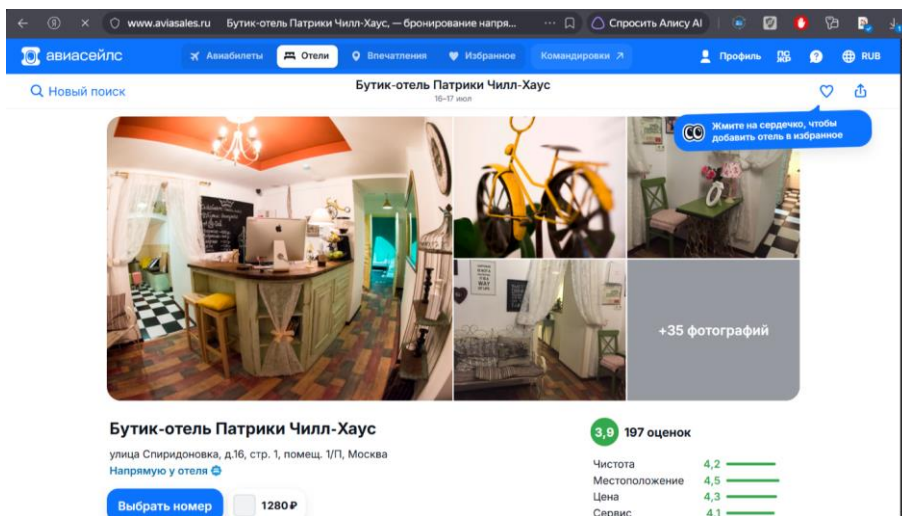


Рисунок 7 – Выбор первого результата

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В процессе разработки автоматизированных UI-тестов были использованы следующие технологии:

1. Язык программирования Java (версия 17) — основной язык разработки тестов.

2. Selenide — библиотека для автоматизации UI-тестов, предоставляющая удобный API для работы с браузером, поиска элементов, ожиданий и выполнения действий. Selenide автоматически управляет WebDriver и упрощает написание стабильных тестов.

3. JUnit 5 — фреймворк для написания и выполнения тестов. Использован для аннотаций `@Test`, `@BeforeEach`, `@AfterEach` и ассертов.

4. Maven — система сборки проекта, управляющая зависимостями и запуском тестов.

5. Logback — библиотека для логирования, используемая для вывода отладочной информации о ходе выполнения тестов.

6. Паттерн Page Object Model — архитектурный паттерн, разделяющий логику тестов, логику страниц и логику элементов. Использован для повышения читаемости и поддерживаемости кода.

7. Git и GitHub — система контроля версий для совместной работы над проектом.

8. Google Chrome — браузер, в котором выполнялись тесты.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на «отлично».

Землякова Анастасия Владимировна, студентка группы 4381, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходила учебную практику по теме “UI-тестирование” с 06.07.2026 по 18.07.2026.

В течение практики обучающаяся Землякова Анастасия Владимировна выполняла разработку автоматизированных UI-тестов для сайта Aviasales и написание тестов каждый на отдельный сценарий использования системы.

В результате практики обучающаяся разработала автоматизированные тесты для выбранного сайта.

По результатам работы Землякова Анастасия Владимировна учебную практику оцениваю на Отлично.

Руководитель практики

Ассистент Кафедры МО ЭВМ 18.06.2026

А.М. Шевелева

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики была достигнута поставленная цель — освоены основные технологии автоматизированного UI-тестирования и разработан набор тестов для веб-приложения Aviasales.

В процессе выполнения работы были изучены теоретические основы UI-тестирования, принципы построения автоматизированных тестов и архитектурный паттерн Page Object Model. Были освоены язык программирования Java, система сборки Maven, библиотека Selenide для взаимодействия с пользовательским интерфейсом и фреймворк JUnit 5 для организации и выполнения тестов. Кроме того, были изучены основы совместной разработки с использованием системы контроля версий Git и платформы GitHub.

В результате практики был разработан чек-лист тестирования и реализовано 10 автоматизированных UI-тестов, охватывающих основные пользовательские сценарии работы с сайтом Aviasales. Созданная архитектура проекта обеспечивает разделение логики тестов, страниц и элементов интерфейса, что делает код более понятным и расширяемым. В рамках работы были реализованы базовые классы, обеспечивающие взаимодействие с элементами пользовательского интерфейса, а также разработаны классы страниц и тестов, соответствующие паттерну Page Object Model.

Полученные результаты подтверждают, что поставленные задачи практики были полностью выполнены. Разработанный набор тестов позволяет автоматизировать проверку наиболее востребованных функций сервиса поиска авиабилетов и может использоваться в дальнейшем развитии системы автоматизированного тестирования

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Selenide: User Guide [Электронный ресурс]. URL: <https://selenide.org/documentation.html> (дата обращения: 12.07.2026).
2. JUnit 5 User Guide [Электронный ресурс]. URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 10.07.2026).
3. Page Object Model [Электронный ресурс]. URL: https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/ (дата обращения: 11.07.2026).

ПРИЛОЖЕНИЕ А